

COMPILER CONSTRUCTION · LAB 8

Syntax-Directed Definitions & Translation

Time Allowed: 90 Minutes

Lab Objective

In this lab, students will:

- Understand the difference between Syntax-Directed Definitions (SDD) and Syntax-Directed Translation (SDT)
- Annotate a context-free grammar with semantic rules to form an SDD
- Identify synthesized and inherited attributes in a grammar
- Construct annotated parse trees and dependency graphs manually
- Trace a full SDT evaluation for an arithmetic expression and compute the result
- Implement a working SDT for expression evaluation in Python

Background

Syntax-Directed Definitions (SDD)

An SDD is a context-free grammar augmented with attributes and semantic rules. Each grammar symbol may carry attributes, and each production has an associated rule that defines how attribute values are computed.

Key terms:

- Synthesized attribute — value computed from children nodes in the parse tree (bottom-up)
- Inherited attribute — value computed from the parent or siblings (top-down)
- Annotated parse tree — parse tree with attribute values at each node
- Dependency graph — directed graph showing how attribute values depend on each other

Syntax-Directed Translation (SDT)

An SDT embeds program fragments (semantic actions) directly inside grammar productions. These actions are executed during parsing, at the point where they appear in the production.

- Actions placed at the end of a production are executed on reduction (S-attributed SDT — compatible with bottom-up parsers)
- Actions placed mid-production are executed during the parse at that point (L-attributed SDT — compatible with top-down parsers)

Grammar with Semantic Rules (SDD)

The following grammar evaluates arithmetic expressions. All attributes are synthesized, making this an S-attributed SDD.

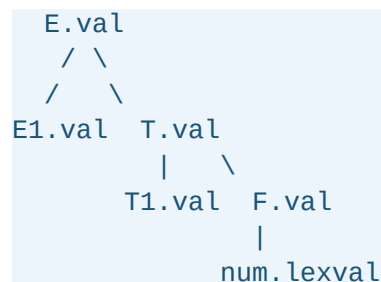
Rule #	Production
1	$E \rightarrow E + T \quad \{ E.val = E1.val + T.val \}$
2	$E \rightarrow T \quad \{ E.val = T.val \}$
3	$T \rightarrow T * F \quad \{ T.val = T1.val * F.val \}$
4	$T \rightarrow F \quad \{ T.val = F.val \}$
5	$F \rightarrow (E) \quad \{ F.val = E.val \}$
6	$F \rightarrow num \quad \{ F.val = num.lexval \}$

Attribute summary:

Symbol	Attribute	Type	Description
E	val	Synthesized	Computed integer value of expression
T	val	Synthesized	Computed integer value of term
F	val	Synthesized	Computed integer value of factor
num	lexval	Synthesized	Integer value from lexer

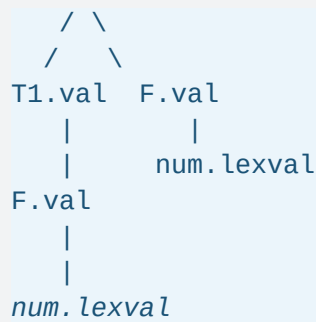
Part A — Dependency Graph

For the production $E \rightarrow E + T$ with rule $E.val = E1.val + T.val$, draw the dependency graph below. Arrows point from the attributes that provide values TO the attribute that depends on them.

Completed example for Rule 1 ($E \rightarrow E + T$):**Task A1 (5 marks):**

Draw the dependency graph for the production $T \rightarrow T * F$ with semantic rule $T.val = T1.val * F.val$.

T.val



Part B — Annotated Parse Tree

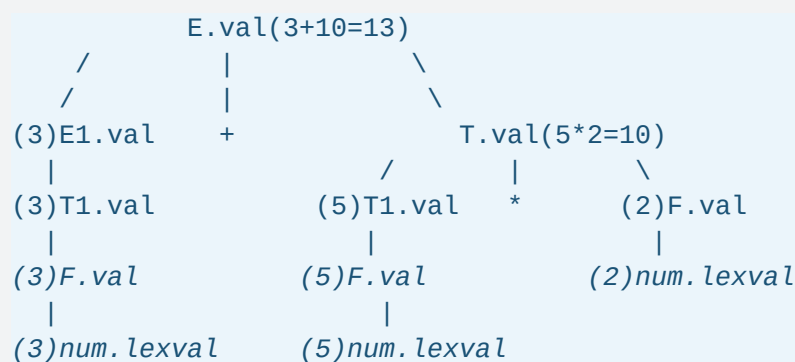
Consider the input string:

3 + 5 * 2

Task B1 (10 marks):

Construct the full annotated parse tree for the expression above. Show all attribute values at each node. Use the grammar rules from the SDD table. Start from the root E.

Hint — parsing order: num(3), F, T, +, num(5), F, T, *, num(2), F, T $\rightarrow$ T*F, E $\rightarrow$ E+T



Part C — Worked Parse & Semantic Trace

Study the following completed trace for: 3 + 5 * 2

The Semantic Stack column shows attribute values computed so far at each reduction step.

Student Name: Musfira(h)

Roll No: 280048443

Step	Stack	Input	Semantic Stack	Action
1	\$	3 + 5 * 2 \$	—	Shift 3
2	\$ num	+ 5 * 2 \$	3	Reduce F → num
3	\$ F	+ 5 * 2 \$	3	Reduce T → F
4	\$ T	+ 5 * 2 \$	3	Reduce E → T
5	\$ E	+ 5 * 2 \$	3	Shift +
6	\$ E +	5 * 2 \$	3	Shift 5
7	\$ E + num	* 2 \$	3, 5	Reduce F → num
8	\$ E + F	* 2 \$	3, 5	Reduce T → F
9	\$ E + T	* 2 \$	3, 5	Shift *
10	\$ E + T *	2 \$	3, 5	Shift 2
11	\$ E + T * num	\$	3, 5, 2	Reduce F → num
12	\$ E + T * F	\$	3, 5, 2	T.val = 5*2 = 10; Reduce T → T*F
13	\$ E + T	\$	3, 10	E.val = 3+10 = 13; Reduce E → E+T
14	\$ E	\$	13	Accept — Result: 13

Final computed result: $E.val = 3 + (5 \times 2) = 3 + 10 = 13$

Part D — Student Exercise (20 Marks)

Using the same grammar and SDD, perform the parse and semantic evaluation for the input string:

(4 + 6) * 3

Instructions:

1. Complete the parse and semantic trace table below. Fill every column for each step.
2. Identify the grammar rule applied at each reduction and write the semantic action.
3. Show the final value of E.val.

Step	Stack	Input	Semantic Stack	Action
1	\$	(4 + 6) * 3 \$	—	Shift (

Student Name: Musfira(h)

Roll No: 280048443

2	\$ (	4 + 3) * 3 \$	(	Shift 4
3	\$ (num	+ 6) * 3 \$	4	Reduce F → num
4	\$ (F	+ 6) * 3 \$	4	Reduce T → F
5	\$ (T	+ 6) * 3 \$	4	Reduce E → T
6	\$ (E	+ 6) * 3 \$	4	Shift +
7	\$ (E +	6) * 3 \$	4	Shift 6
8	\$ (E + num	) * 3 \$	4, 6	Reduce F → num
9	\$ (E + F	) * 3 \$	4, 6	Reduce T → F
10	\$ (E + F	) * 3 \$	4, 6	Reduce E → E + T
11	\$ (E	) * 3 \$	10	Shift)
12	\$ (E)	* 3 \$	10	Reduce F → (E)
13	\$ F	* 3 \$	10	Reduce T → F
14	\$ T	* 3 \$	10	Shift *
15	\$ T *	3 \$	10	Shift 3
16	\$ T * num	\$	10, 3	Reduce F → num
17	\$ T * F	\$	10, 3	Reduce T → T * F
18	\$ T	\$	30	Reduce E → T
19	\$ E	\$	30	Accept Result: 30

Final answer: **E.val = (4 + 6) * 3 = 10 * 3 = 30**

Lab Summary & Key Concepts

Concept	Key Point
SDD	CFG + attributes + semantic rules — defines WHAT to compute
SDT	CFG + embedded actions — defines WHEN to compute during parsing
Synthesized attribute	Flows bottom-up (from children to parent)
Inherited attribute	Flows top-down (from parent or siblings to child)

Student Name: Musfira(h) Roll No: 280048443

S-attributed SDD	Only synthesized attributes — compatible with LR parsers
L-attributed SDD	Synthesized + left-sibling inherited — compatible with LL parsers
Dependency graph	Shows computation order; must be acyclic for SDD to be well-defined
Annotated parse tree	Parse tree with attribute values evaluated at each node